

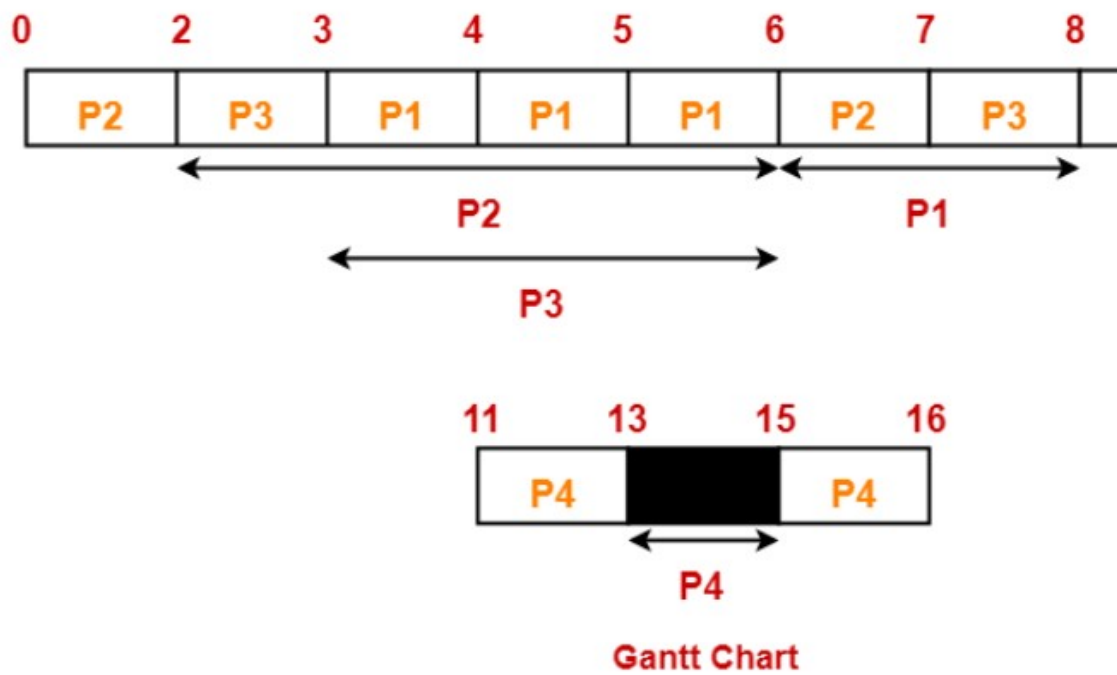
Consider three CPU-intensive processes, which require 10 ,20 and 30 time units and arrive at times 0, 2 and 6 respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end.

Consider the set of 4 processes whose arrival time and burst time are given below-

Process No.	Arrival Time	Burst Time		
		CPU Burst	I/O Burst	
P1	0	3	2	
P2	0	2	4	
P3	2	1	3	

If the CPU scheduling policy is Shortest Remaining Time First, calculate the average waiting time and average turn around time.

Gantt Chart-



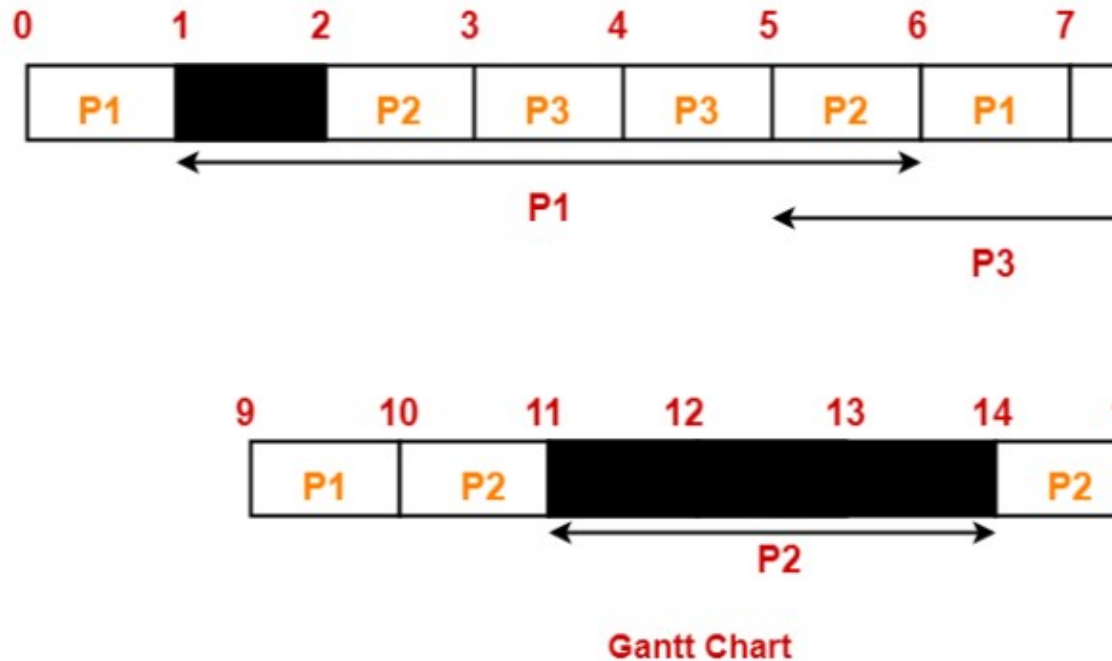
Process Id	Exit time	Turn Around time	Waiting time
P1	11	$11 - 0 = 11$	$11 - (3 + 3) = 5$
P2	7	$7 - 0 = 7$	$7 - (2 + 2) = 3$
P3	9	$9 - 2 = 7$	$7 - (1 + 1) = 5$
P4	16	$16 - 5 = 11$	$11 - (2 + 2) = 7$

Now,

Consider the set of 4 processes whose arrival time and burst time are given below-

Process No.	Arrival Time	Priority	Burst Time	
			CPU Burst	I/O Burst
P1	0	2	1	5
P2	2	3	3	3

If the CPU scheduling policy is Priority Scheduling, calculate the average waiting time and average turn around time. (Lower number means higher priority)



Now, we know-

- Turn Around time = Exit time – Arrival time
- Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting
------------	-----------	------------------	---------

- Average Turn Around time = $(10 + 13 + 6) / 3 = 29 / 3 = 9.67$ units
- Average waiting time = $(6 + 9 + 3) / 3 = 18 / 3 = 6$ units

Consider the following pseudocode, where S is a semaphore initialized to 5 in line 2 and counter is a shared variable initialized to 0 in line 1. Assume that the increment operation in line 7 is atomic.

```

1.  int counter = 0;
2.  Semaphore S = init(5);
3.  void parop(void)
4.  {
5.      wait(S);
6.      wait(S);

```

```

7.         counter++;
8.         signal(S);
9.         signal(S);
10.    }

```

If five threads execute the function concurrently, which of the following program behavior(s) is/are possible?

- A. The value of counter is 5 after all the threads successfully complete the execution
 - B. The value of counter is 1 after all the threads successfully complete the execution
 - C. The value of counter is 0 after all the threads successfully complete the execution
 - D. There is a deadlock involving all the threads
-
- A. Assume that the threads execute sequentially with no interleaving then after each thread ends the counter value increments by 1. Hence after threads finish, counter value will be incremented times from to **Possible**.
 - B. Let's assume that a process used 2 waits and reads the counter value and didn't update the value yet, all the other process let's say the other processes executed sequentially incremented and stored the value as 4 but since the value isn't written the first process yet the current value is overwritten by the first process as 1. **Possible**
 - C. There exists no pattern of execution in which the process increments the current value and completes while maintaining 0 as the counter value. **Not possible**
 - D. Assume that all the process use up the first wait operation, the semaphore value will now become zero and deadlock would've occurred. **Possible**

```

*****
*****
*****

```

Which one or more of the following need to be saved on a context switch from one thread (T1) of a process to another thread (T2) of the same process?

- (A) Page table base register
- (B) Stack pointer
- (C) Program counter
- (D) General purpose registers

Correct Answer: (B), (C), and (D)

Solution:

Understanding context switch When switching from one thread (T1) to another (T2) within the same process, the CPU must save the current execution state of T1 and load the execution state of T2. This execution state primarily consists of CPU registers and pointers required to resume execution later.

Evaluating each option-

(A) Page table base register: All threads of the same process share the same virtual address space, hence the same page table base register. Therefore, it does not need to be saved or changed during a thread-to-thread context switch. ⇒Not required.

(B) Stack pointer: Each thread maintains its own stack (for function calls, local variables, return addresses). To resume T2 correctly, its stack pointer must be restored. ⇒Must be saved / restored.

(C) Program counter: The program counter (PC) holds the address of the next instruction to execute. Without saving/restoring the PC, the thread cannot resume from where it left off. ⇒Must be saved / restored.-

(D) General purpose registers: General purpose registers (like AX, BX in x86, or r0, r1 in ARM) hold intermediate data during computation. Each thread requires its own register values to resume correctly. ⇒Must be saved / restored.

Conclusion: Only the thread-specific state (stack pointer, program counter, and general registers) needs to be saved / restored. The address-space-related state (page table base register) remains unchanged because threads of a process share memory.

Which one or more of the following options guarantee that a computer system will transition from user mode to kernel mode?

- (A) Function Call
- (B) malloc Call
- (C) Page Fault
- (D) System Call

Correct Answer: (C) and (D)

Solution:

Analyze Option (A) Function Call: A normal function call in user space executes entirely in user mode. It does not involve the operating system kernel. Hence, there is no user-to-kernel mode switch. ⇒(A)is False.

Analyze Option (B) malloc Call The malloc() function is part of the C standard library. In most cases, it only allocates memory from the heap of the process without involving the kernel. However, if more memory is required, it may internally invoke a system call such

as `brk()` or `mmap()`, which would cause a kernel transition. But this is not guaranteed for every `malloc` call. $\Rightarrow$ (B) is False.

Analyze Option (C) Page Fault: A page fault occurs when a program accesses a memory page that is not present in physical memory. This is a hardware exception, which always triggers the operating system's page fault handler in the kernel. Thus, the CPU must switch to kernel mode to handle it. $\Rightarrow$ (C) is True.

Analyze Option (D) System Call System calls are explicit requests made by a user process to the operating system for services like I/O, process management, or memory management. Executing a system call instruction (e.g., `int 0x80`, `syscall`) always causes a trap into kernel mode. $\Rightarrow$ (D) is True.

Which one or more of the following CPU scheduling algorithms can potentially cause starvation?

(A) First-in First-Out

(B) Round Robin

(C) Priority Scheduling

(D) Shortest Job First

Round Robin (RR) scheduling will **never lead to starvation** because every job is guaranteed CPU time after a fixed time quantum.

Since the time quantum is finite, each process eventually gets its turn to execute after a finite period, ensuring fairness among all tasks.

It is worth noting that the time quantum can be very large — for example, 10 minutes, 10 years, or even 100 years — but as long as it remains finite, **every process will still receive CPU time** and starvation will not occur.

Can First Come First Serve lead to Starvation?

Just imagine a first job submitted to CPU is –

```
while(1);
```

Yes, Infinite loop. Do you think other jobs will ever get a chance?

```
*****
*****
*****
```

Consider the two functions incr and decr shown below.

```
incr(){
wait(s);
X = X+1;
signal(s);
}

decr(){
wait(s);
X = X-1;
signal(s);
}
```

There are 5 threads each invoking incr once, and 3 threads each invoking decr once, on the same shared variable X. The initial value of X is 10.

Suppose there are two implementations of the semaphore s, as follows:

I-1: s is a binary semaphore initialized to 1.

I-2: s is a counting semaphore initialized to 2.

Let V1, and V2 be the values of X at the end of the execution of all the threads with implementations I-1, and I-2, respectively.

Which one of the following choices corresponds to the minimum possible values of V1, V2, respectively?

- A. 15,7
- B. 7,7
- C. 12,7
- D. 12,8

I==> S is a binary semaphore initialized to 1

as s is a binary semaphore it can take only two values 0 & 1.

now let's see the code

```
incr(){
    wait(s);
    X = X+1;
    signal(s);
}

decr(){
    wait(s);
    X = X-1;
    signal(s);
}
```

so code is simple increment by 1 and decrement by 1 is done.

Now it is mentioned in the question there are 5 inc and 3 dec

and initial value of x is 10

now it is asking us the min value we can have after execution of 5 inc and 3 dec operation on X.

so think as s=1 it mean either you enter inc or decr code you will not able to enter the other at the same time you have to only do one at a time .

so whatever order of sequence you implement you know that you are doing 5 inc and 3 decr at all in the value of x

so final min value of $x = x + 5 - 3 = 10 + 5 - 2 = 12$ ans .

Now look at the second method using Counting semaphore

```
incr(){                                decr(){
    wait(s);                            wait(s);
    X = X+1;                            X = X-1;
    signal(s);                          signal(s);
}
```

and $s=2$ and $x=10$

now as $s=2$ it means you can enter in both the inc or decr block at the same time and access the critical section of the code at the same time >

ok let's see it

you know that inc works

as 1. first value of x is stored in a r1

2. value of r1 is inc by 1

3. value of r1 is stored in x

similarly decrement works the same way

1. first value of x is stored in a r2

2. value of r2 is dec by 1

3. value of r2 is stored in x

Now suppose both enter into inc and decr at the same time executed wait so $s=0$

now both access the shared variable i.e critical section

now suppose

in inc $r1=x=10$

in dec $r2=x=10$

now 5 inc operation continuously followed out so $x=15$

now we know we also have to run decr and we have executed first decr until $r2=x=10$

now 2nd step of decr will have so $r2=9$

now r2 is stored in x

so 15 is replaced by 9

and remaining 2 dec operation will carry on and make the value of x =7(min).

Each of a set of n processes executes the following code using two semaphores a and b initialized to 1 and 0, respectively. Assume that count is a shared variable initialized to 0 and not used in CODE SECTION P.

CODE SECTION P

```
wait(a); count=count+1;  
if (count==n) signal (b);  
signal (a); wait (b) ; signal
```

What does the code achieve?

- A. It ensures that no process executes CODE SECTION Q before every process has finished CODE SECTION P.
- B. It ensures that atmost two processes are in CODE SECTION Q at any time.
- C. It ensures that all processes execute CODE SECTION P mutually exclusively.
- D. It ensures that at most n-1 processes are in CODE SECTION P at any time.

Stepwise Explanation:

Initialization: $a=1, b=0, \text{count}=0$

There are n processes say, $P_1, P_2, P_3, \dots, P_n$.

Let's assume P_1 has executed successfully the code section P and encounters the statements:

wait (a);

Now, 'a' becomes 0, so all the subsequent processes are blocked.

$b=0, \text{count}=0$.

..... $\text{count}=1$ $a=0, b=0, \text{count}=1$

signal (a); [$a=1, b=0, \text{count}=1$. Other processes can now execute *Wait(a)*]

wait (b) ; [$b=-1$, therefore P_1 also gets blocked. This statement would not be executed]

signal (b);

This statement would only run if its preceding statement is executed. In turn, it would only be executed if $(\text{count}==n)$. Therefore, no process would execute Q until every process has successfully executed P .

Concurrency and synchronization

<https://www.geeksforgeeks.org/quizzes/gate-jpc-concurrency-and-synchronization-deadlock-pyq-2010-to2025/>

.....

.....

-
- Paging is a non-contiguous memory allocation technique.
 - **Page Table** is a data structure that performs the mapping of page number to the frame number.

Important Formulas-

The following list of formulas is very useful for solving the numerical problems based on paging.

For Main Memory-

- Physical Address Space = Size of main memory
- Size of main memory = Total number of frames x Page size
- Frame size = Page size
- If number of frames in main memory = 2^X , then number of bits in frame number = X bits
- If Page size = 2^X Bytes, then number of bits in page offset = X bits
- If size of main memory = 2^X Bytes, then number of bits in physical address = X bits

For Process-

- Virtual Address Space = Size of process
- Number of pages the process is divided = Process size / Page size
- If process size = 2^X bytes, then number of bits in virtual address space = X bits

For Page Table-

- Size of page table = Number of entries in page table x Page table entry size
- Number of entries in pages table = Number of pages the process is divided
- Page table entry size = Number of bits in frame number + Number of bits used for optional fields if any

NOTE-

- In general, if the given address consists of 'n' bits, then using 'n' bits, 2^n locations are possible.
- Then, size of memory = $2^n \times$ Size of one location.
- If the memory is byte-addressable, then size of one location = 1 byte.
- Thus, size of memory = 2^n bytes.
- If the memory is word-addressable where 1 word = m bytes, then size of one location = m bytes.
- Thus, size of memory = $2^n \times m$ bytes.

PRACTICE PROBLEMS BASED ON PAGING AND PAGE TABLE-

Problem-01:

Calculate the size of memory if its address consists of 22 bits and the memory is 2-byte addressable.